

Inconsistencias de Datos — Diagnóstico y Plan

Para: Magdalena Ortega **De:** David Cabezas · Value Strategy Consulting **Fecha:** 6 de mayo, 2026 **Sistema:** mageam.com (producción) · cliente Goldfields Salares Norte

Resumen Ejecutivo

Tras auditoría forense de la base de datos productiva, confirmamos **una sola inconsistencia crítica** y descartamos otras nueve verificaciones. La causa es arquitectónica y previsible: el modelo de datos usa **strings flotantes** en lugar de claves foráneas, y **SQLite no valida claves foráneas por default**.

Indicador	Valor
OTs huérfanas confirmadas	124
<code>equipment_tags</code> distintos sin match en jerarquía	32
Tags con sugerencia automática $\geq$ 90% confianza	4
Tags sin sugerencia (cancelar / investigar)	2
Verificaciones que pasaron OK	9 de 9

Decisión clave previa a elegir cloud: ¿seguimos en SQLite o migramos a Postgres? Esa decisión define la propuesta de plataforma (GCP / AWS / Azure).

1. Estado actual de la base productiva

Tabla	Filas
Plants	4
HierarchyNodes (equipos + jerarquía)	10.136
WorkRequests (avisos)	171
ManagedWorkOrders (OTs)	138
Workforce (técnicos)	35
AuditLog	403

2. Inconsistencias detectadas

2.1 — 124 OTs huérfanas

32 `equipment_tag` distintos en `managed_work_orders` no existen en `hierarchy_nodes.tag`.

Impacto operativo:

- Los buscadores muestran las OTs pero no encuentran el equipo asociado.
- Los KPIs por equipo no las cuentan.
- Cualquier `JOIN` entre OTs y jerarquía pierde datos.

Distribución por estado:

Status	OTs afectadas
CERRADO	47
EN_PROGRAMACION	22
PROGRAMADO	21
EN_EJECUCION	19
CANCELADO	8
Otros	7

Top 5 tags huérfanos:

Tag huérfano	OTs	Sugerencia auto (score)
<code>PMP-SL-HP-002</code>	10	<code>PMP-SLP-001</code> (83%)
<code>COM-AIRE-02</code>	6	sin match fuerte
<code>PMP-AGUA-01</code>	6	<code>PMP-SLP-001</code> (64%)
<code>CRU-CON-HP-01</code>	6	<code>CSP-001</code> (58%)
<code>THK-REL-02</code>	6	sin match fuerte

Causa raíz operativa: la limpieza SF-634 (eliminación de plantas duplicadas) borró 32 nodos del subárbol "Goldfields Salares Norte". Las OTs históricas que apuntaban a ellos por `equipment_tag` (string) quedaron huérfanas porque la referencia no era una FK que SQLite pudiera validar.

2.2 — Verificaciones que pasaron OK

Verificación	Resultado
WRs sin equipment_tag	0
OTs sin equipment_tag	0
OTs con <code>work_request_id</code> huérfano	0
Plant references inválidas	0
Status no canónicos en OTs	0
Técnicos sin specialty	0
Nodos con parent huérfano	0
Tags duplicados en misma planta	0
Case drift en tags	0

3. Causa raíz arquitectónica

El modelo

```
class ManagedWorkOrderModel(Base):
    equipment_tag: Mapped[str] # ← string libre
    equipment_id: Mapped[str] # ← string libre, NO es FK
    work_request_id: Mapped[str | None] = ForeignKey(...) # ← SI es FK
```

Lo que pasa hoy

1. La OT guarda `equipment_tag = "PMP-SL-HP-002"` como **string suelto**, sin FK al hierarchy.
2. SQLite **no valida foreign keys por default** (necesita `PRAGMA foreign_keys=ON` por conexión).
3. Si alguien borra un nodo, las OTs siguen referenciándolo por nombre.
4. El frontend hace lookup por string → no encuentra → muestra el tag pero sin contexto.

Por qué pasó

- **Diseño legacy:** los equipos llegaron por importación CSV/Excel sin jerarquía real. El string era todo lo que había.
- **Hierarchy se agregó después** sin migrar las OTs.
- **Cleanup operacional** (SF-634) borró duplicados sin migrar referencias.

4. Soluciones (3 opciones, ordenadas por costo)

Opción A — Mínima (1 día)

Limpieza puntual con script. **Ya disponible:** `scripts/audit_orphan_equipment.py` + `scripts/suggest_orphan_mapping.py` generan dos CSVs:

- `orphan_tags.csv` : 32 tags + breakdown por estado.
- `orphan_tags_suggested.csv` : top-3 candidatos por similitud.

Jorge revisa, confirma el mapeo o marca `CANCELAR` . Aplicamos UPDATE batch.

Pro	Contra
Rápido, no toca arquitectura	No previene el próximo caso
4 mapeos $\geq$ 90% son auto-aplicables	Vuelve a pasar la próxima limpieza

Opción B — Defensiva (3 días)

Opción A + activar `PRAGMA foreign_keys=ON` en cada conexión SQLite + migrar `equipment_id` a FK real con `ON DELETE RESTRICT` + tests de integridad.

Pro	Contra
Previene futuras roturas	SQLite sigue limitando a escala
Sin cambiar plataforma	Sin concurrencia real ni replication

Opción C — Postgres en cloud (1 semana)

Opción B + migración SQLite → Postgres managed + FK enforcement nativo + check constraints + backups automáticos + point-in-time recovery + concurrencia real.

Pro	Contra
Arquitectura correcta para escala (50K equipos, 100K OTs/año proyectado)	1 semana de migración
Backups y recovery sin esfuerzo	Costo cloud (~\$50–150/mes)

5. Comparación de plataformas (sólo aplica si vamos a Opción C)

Criterio	Peso	GCP	AWS	Azure
Datacenter en Chile (latencia)	Alto	✓ Santiago	X São Paulo	X São Paulo
Compliance ISO 27001 / minero	Alto	✓	✓	✓
Postgres managed	Alto	Cloud SQL	RDS	Azure DB
Object storage (DMS, fotos)	Medio	GCS	S3	Blob
Vector DB (RAG futuro)	Medio	Vertex AI Search	OpenSearch / pgvector	AI Search
Anthropic Claude nativo	Alto	via API	Bedrock nativo	X (sólo GPT)
Costo mensual estimado	Alto	\$280–400	\$320–450	\$300–430
Curva aprendizaje VSC	Medio	Moderada	Alta	Moderada
Free tier inicial	Bajo	\$300 / 90d	\$200 / 12m	\$200 / 30d

Recomendación preliminar

GCP por:

- 1. Cloud SQL en Santiago = latencia <30 ms desde Salares Norte (Atacama).
- 2. Vertex AI Search incluye RAG gestionado (ahorra ~80% del trabajo del RAG futuro).
- 3. Costo competitivo.
- 4. Soberanía de datos en Chile (requisito habitual minería).

AWS preferido SI el cliente exige Bedrock nativo o ya tiene contrato corporativo. **Azure** queda tercero salvo Microsoft-shop.

Anexos

Anexo A — Scripts de auditoría (re-ejecutables)

Ambos scripts están en `scripts/` y son sólo-lectura.

1. Detectar huérfanos

```
docker cp scripts/audit_orphan_equipment.py ocp-backend:/tmp/audit.py
docker exec ocp-backend python /tmp/audit.py \
  /app/data/ocp_maintenance.db \
  /app/data/audit_output
```

Genera `orphan_tags.csv` (32 tags + breakdown) y `orphan_wos.csv` (124 OTs detalle).

2. Sugerir mapeos por similitud

```
docker cp scripts/suggest_orphan_mapping.py ocp-backend:/tmp/sugg.py
docker exec ocp-backend python /tmp/sugg.py \
  /app/data/audit_output/orphan_tags.csv \
  /app/data/ocp_maintenance.db \
  /app/data/audit_output/orphan_tags_suggested.csv
```

Rellena la columna `mapeo_propuesto` con top-3 candidatos por SequenceMatcher ≥ 0.55 .

Resultado producción (06-may-2026): 30 / 32 tags con sugerencia, 2 sin match. 4 tags con confianza $\geq 90\%$ (auto-aplicables sin revisión humana).

Anexo B — Claude API: tres formas de consumirlo

Tema relevante para la decisión de cloud.

Modo	Endpoint	Datos viajan	Latencia	Facturación
API directo (hoy)	<code>api.anthropic.com</code>	A servidores Anthropic en US	Variable	Factura Anthropic separada
AWS Bedrock	Endpoint AWS interno	Quedan en tu cuenta AWS	Predecible (red AWS)	Junto con factura AWS
GCP Vertex AI	Endpoint Google interno	Quedan en tu cuenta GCP	Predecible (red Google)	Junto con factura GCP
Azure OpenAI	(Azure NO tiene Claude)	—	—	—

Cuándo importa "Claude nativo"

- **Compliance:** datos no pueden cruzar a otro proveedor.
- **Single vendor:** una factura, un soporte, un SLA.
- **VPC privado:** acceso vía VPC endpoint sin pasar por internet pública.
- **Logs centralizados:** CloudWatch o Cloud Logging integrados.

Cuándo NO importa

- **Pricing:** API directo es 10–15% más barato (sin markup del cloud).
- **Velocidad de adopción:** API directo se conecta en 5 minutos.
- **Estado actual mageam.com:** API directo y funciona perfecto.